基于强化学习的贪吃蛇游戏 —— 软件详细设计文档

> **适用对象**：学习 Python 编程、面向对象（OOP）、Pygame 游戏开发及深度强化学习（DQN）的初学者
> **配套代码**： `enhancedSnake.py` （核心实现）、 `demo_rl_snake.py` （模型演示入口）
> **学习目标**：通过"经典贪吃蛇 + AI 自动控制"这一完整项目，循序渐进掌握 **程序结构** → **面向对象** → **事件循环** → **强化学习** 的知识链

文档说明与阅读指引

本文档在经典贪吃蛇项目的基础上引入 **深度 Q 网络（DQN）**，使"由学生键盘控制"升级为"AI 自动学习控制"。为符合学生认知规律，章节按 **由浅入深、由整体到局部** 的顺序组织：

1. 先建立**整体印象**（项目概述、程序结构）；
2. 再掌握**基础知识**（配置、游戏环境、Pygame 绘制）；
3. 最后进阶**强化学习核心**（状态、奖励、DQN、经验回放、训练流程）。

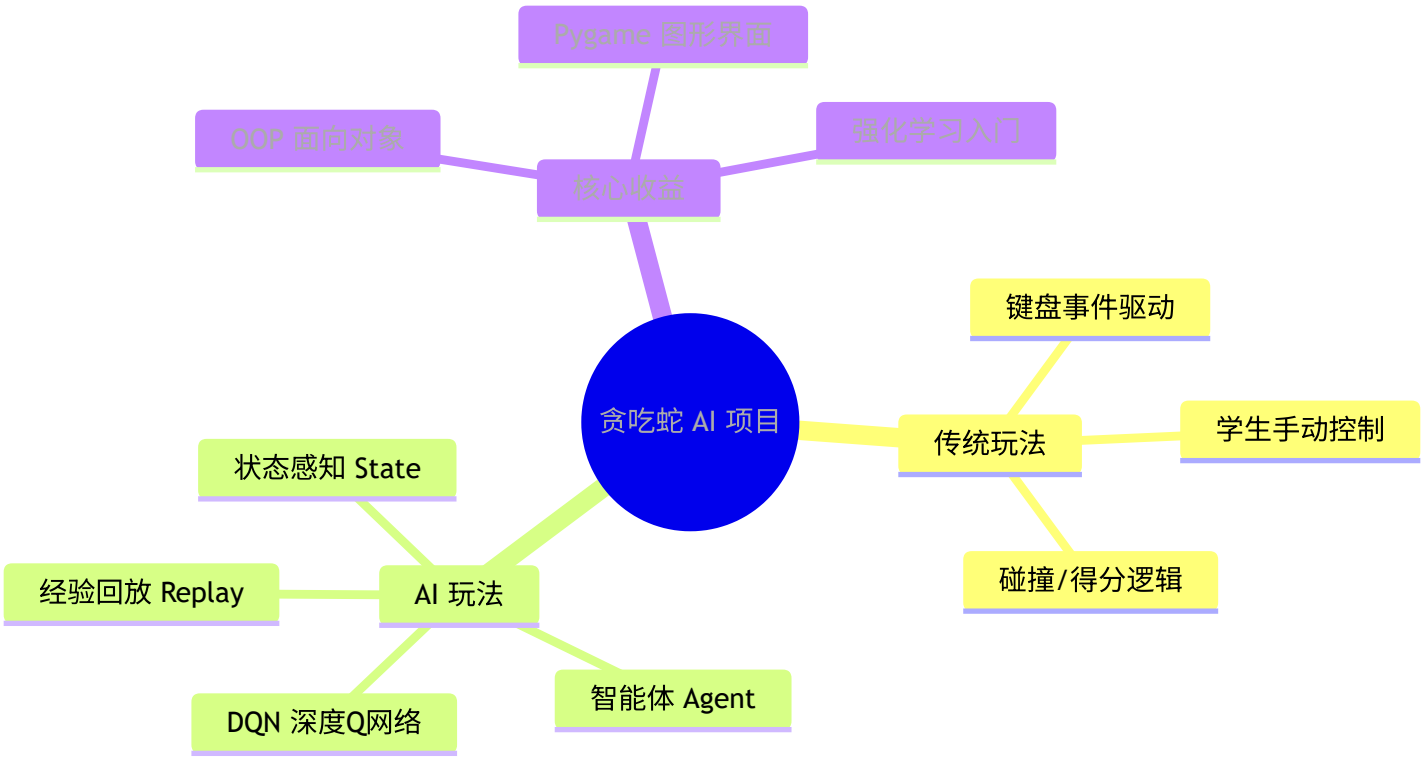
> 💡 建议：初次阅读可先跳过第 7、8 章的强化学习公式，待理解游戏框架后再回头研读。

1. 项目概述

1.1 项目背景

贪吃蛇是编程学习的"经典入门项目"，涵盖 **面向对象、事件驱动、碰撞检测、界面绘制** 等核心知识点。本项目在此基础上进一步引入 **深度强化学习（Deep Q-Network, DQN）**，让程序能够：

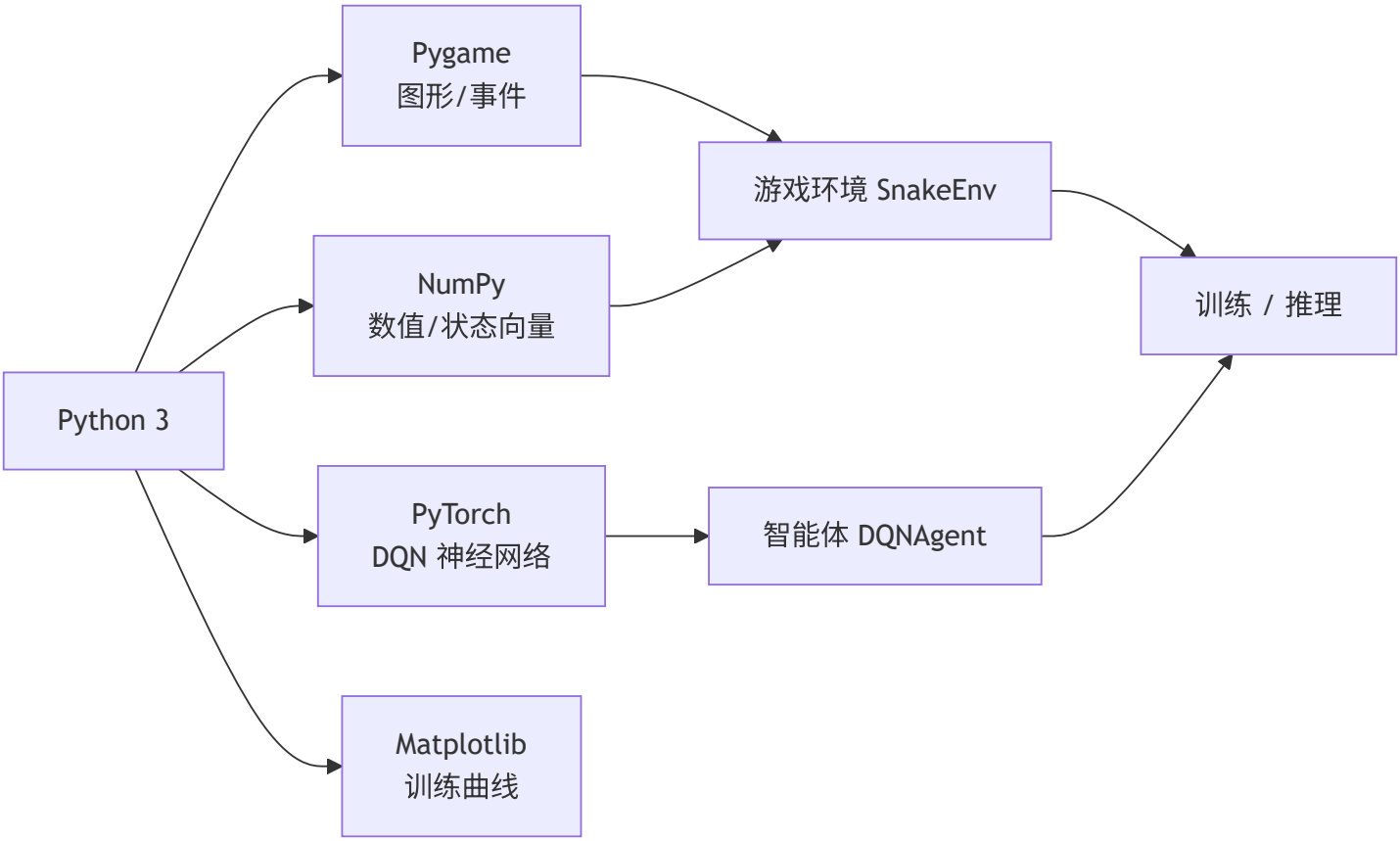
- **阶段 A（传统模式）**：学生用键盘方向键控制蛇移动、吃食物、躲避障碍；
- **阶段 B（AI 模式）**：智能体（Agent）通过"试错—学习—优化"自动学会吃食物、躲避碰撞，并在训练后自主完成游戏。



1.2 项目目标

编号	目标	对应知识点
G1	实现可运行的贪吃蛇游戏（移动、吃食物、死亡判定）	游戏主循环、碰撞检测
G2	采用面向对象结构，职责清晰	类、封装、组合
G3	支持键盘控制（传统）与 AI 控制（强化学习）双模式	事件驱动、策略模式
G4	用 DQN 训练智能体并可视化训练曲线	神经网络、强化学习

1.3 技术栈



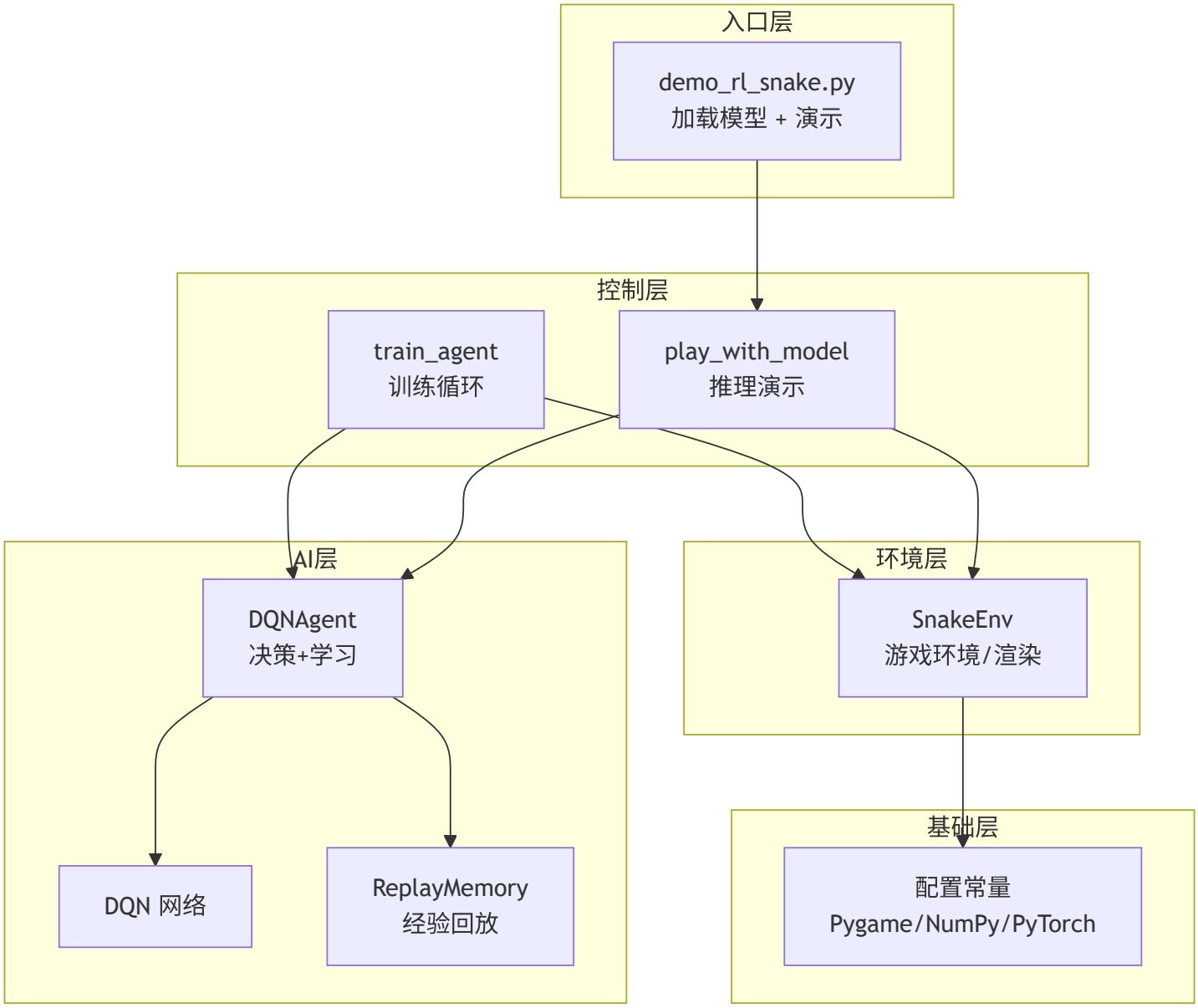
2. 程序结构与文件说明

2.1 文件清单

参考经典贪吃蛇的分层结构，本项目虽集中在单文件中便于教学，但**逻辑上仍可拆分为清晰的职责模块**（对应 PDF 设计文档的 `main / game / snake / food / config` 分层思想）：

文件	角色（对应经典分层）	主要职责
`config` （全局常量区）	`config.py`	屏幕、网格、颜色、方向、DQN 超参数
`SnakeEnv` 类	`snake.py + food.py + game.py`	蛇与食物状态、移动、碰撞、绘制
`DQN` / `ReplayMemory` / `DQNAgent` 类	（AI 扩展层）	神经网络、经验回放、决策与学习

文件	角色（对应经典分层）	主要职责
`train_agent()` / `play_with_model()`	`game.py` 主循环	训练流程、模型推理演示
`demo_rl_snake.py`	`main.py` 入口	加载模型并可视化演示



2.2 目录结构（推荐拆分方式）

```
snakeRL/
├── config.py            # 全局配置（常量）
├── environment.py        # SnakeEnv 游戏环境
├── agent.py              # DQN / ReplayMemory / DQNAgent
├── train.py              # train_agent 训练流程
├── enhancedSnake.py      # 当前整合版（教学用，单文件）
└── demo_rl_snake.py      # 演示入口
```

> 📌 当前 `enhancedSnake.py` 将所有模块整合在单文件中，**降低初学者的文件跳转成本**；熟悉结构后，可参照上表拆分为多文件工程。

3. 配置设计（config）

将所有"魔法数字"集中定义为常量，提高可读性与可维护性。

```python
SCREEN_WIDTH, SCREEN_HEIGHT = 400, 400
GRID_SIZE = 20                      # 每个网格像素大小
GRID_WIDTH  = SCREEN_WIDTH // GRID_SIZE
GRID_HEIGHT = SCREEN_HEIGHT // GRID_SIZE
FPS = 30

# 方向常量（用元组表示位移，便于向量运算）
UP, DOWN, LEFT, RIGHT = (0,-1), (0,1), (-1,0), (1,0)

# DQN 超参数
LEARNING_RATE = 0.001
GAMMA = 0.99                        # 折扣因子
BATCH_SIZE = 64
MEMORY_CAPACITY = 10000
EPSILON_START, EPSILON_END, EPSILON_DECAY = 1.0, 0.01, 0.995
TARGET_UPDATE = 100                 # 目标网络更新频率
```

知识点说明：

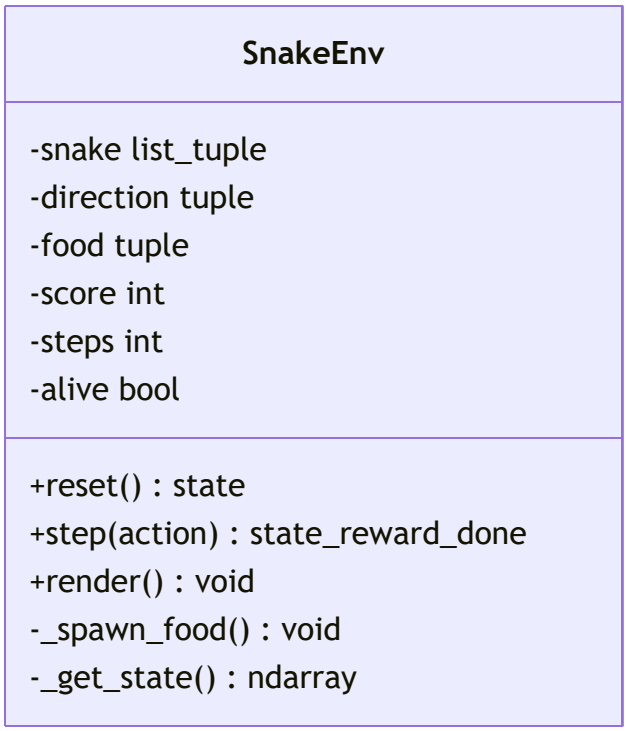
- **常量集中管理**：修改屏幕大小、难度只需改一处，避免"硬编码散落"。
- **方向用元组**：`(dx, dy)` 可直接与坐标相加，实现"向量式移动"。



4. 游戏环境设计（SnakeEnv）

`SnakeEnv` 是项目的**核心类**，负责"游戏世界"的全部规则：蛇、食物、移动、碰撞、绘制。它同时遵循**强化学习环境接口**（reset / step），可被智能体调用。

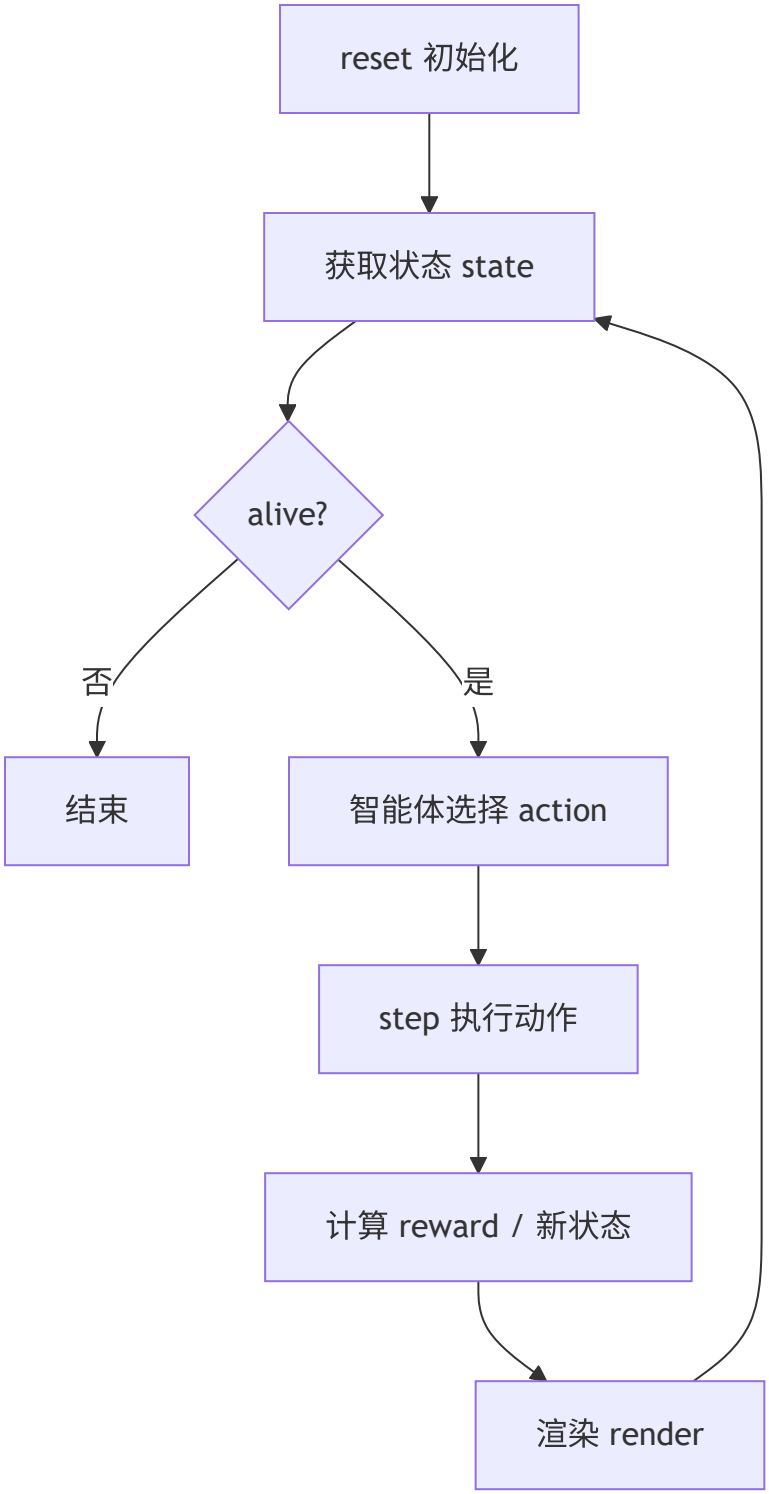
4.1 类结构



4.2 核心属性

属性	类型	说明
`snake`	`list[(x,y)]`	蛇身坐标列表，**索引 0 为蛇头**
`direction`	`tuple`	当前移动方向
`food`	`tuple`	食物坐标
`score`	`int`	当前得分（吃到食物 +1）
`steps`	`int`	已走步数（用于防无限循环）
`alive`	`bool`	是否存活

4.3 主循环流程（算法）



关键方法说明：

- **`reset()`**：重置蛇身（初始长度 3）、方向、分数、食物，返回初始状态。
- **`step(action)`**：强化学习的核心接口，执行一步并返回 `(next_state, reward, done)`。
- **`render()`**：用 Pygame 绘制网格、蛇、食物、分数、步数。

4.4 碰撞检测逻辑



> 🎓 **知识点**：碰撞检测本质是"坐标判定"——边界检测用范围比较，自身碰撞用 `in` 列表判断，食物判定用坐标相等比较。

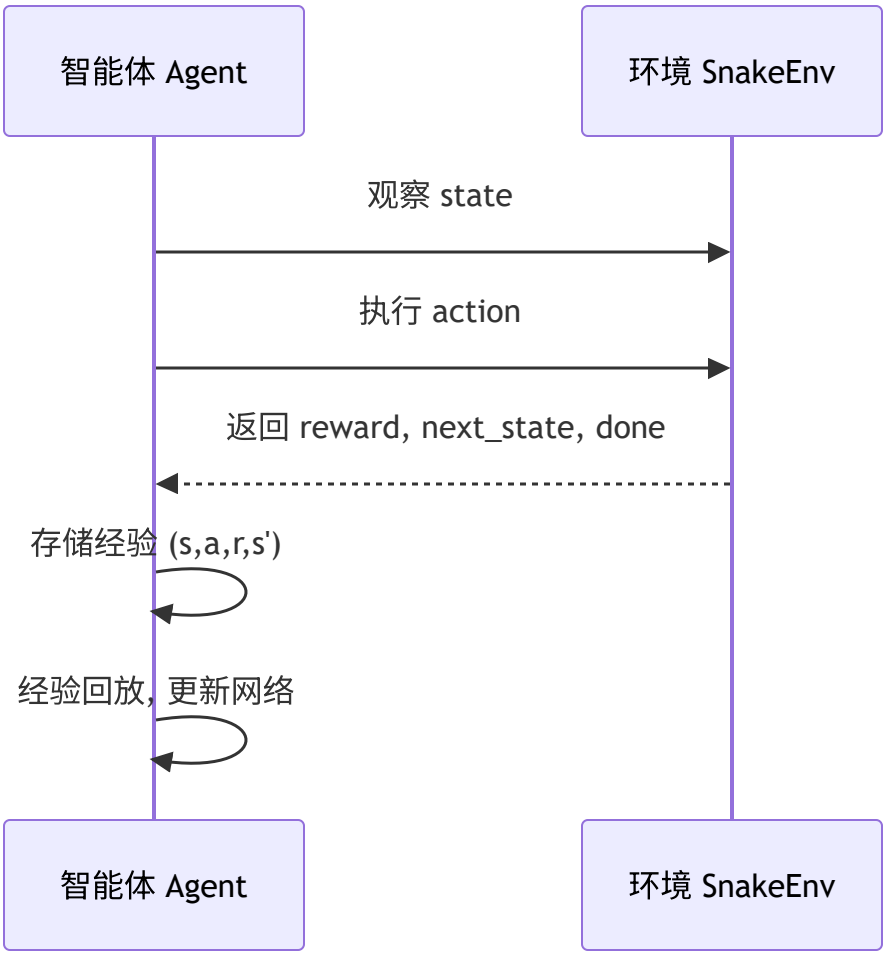
5. 强化学习核心设计 ⭐

> 本章是本项目与传统贪吃蛇的**最大差异**，也是进阶重点。先建立"智能体如何学习"的直觉，再深入公式。

5.1 核心思想：试错学习

智能体（Agent）并不知道"怎么走才对"，它通过大量尝试：

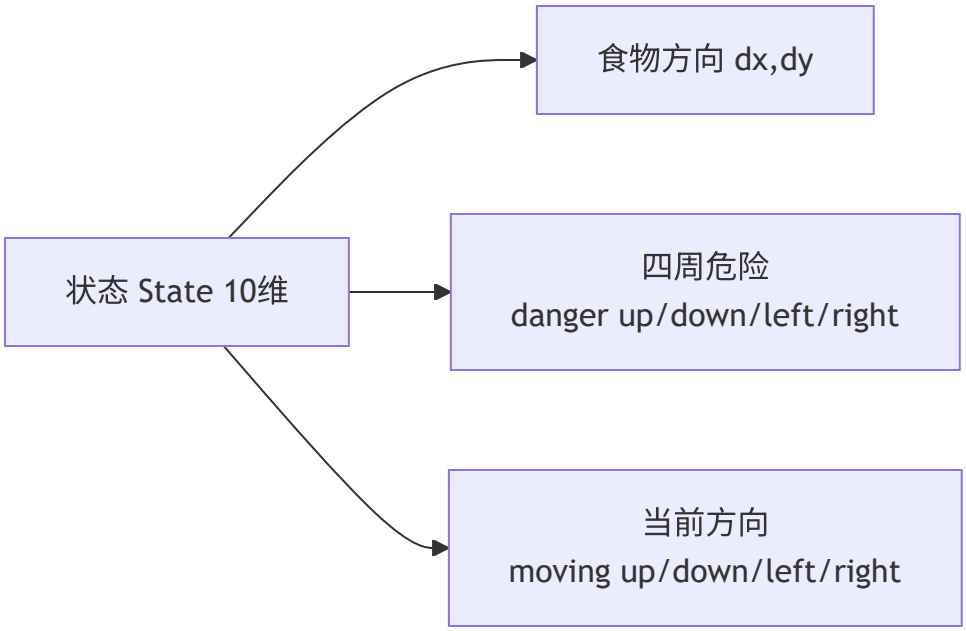
1. **观察**当前状态（state）—— 如"食物在右上方、左边有墙"；
2. **执行**动作（action）—— 上/下/左/右；
3. **获得**奖励（reward）—— 吃食物得分、撞墙受罚；
4. 据此**调整策略**，让未来获得的总奖励尽量高。



5.2 状态空间（State）

状态是智能体的"眼睛"。本项目用 **10 维向量**描述当前局面：

维度	含义
1–2	食物相对蛇头的方向（归一化 dx, dy）
3–6	上/下/左/右 **是否危险**（墙或自身，0/1）
7–10	当前移动方向（one-hot：上/下/左/右）



> 💡 **奖励塑形（Reward Shaping）**：除"吃食物 +20、死亡 -20"外，还按"靠近食物 +1、远离 -1、停滞 -0.1"引导蛇主动接近食物，加速学习。

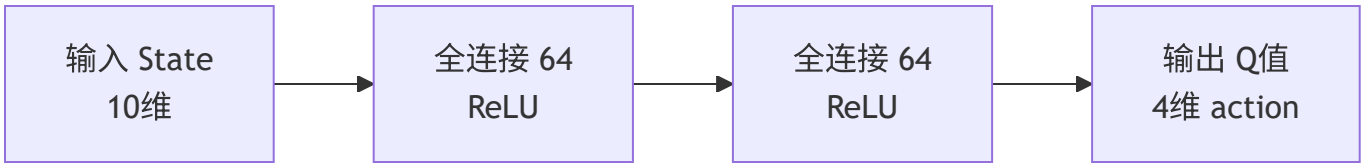
5.3 动作与奖励

动作 action	0	1	2	3
方向	上	下	左	右

情形	奖励	设计意图
吃到食物	+20	强烈正向
撞墙/撞身	-20	强烈惩罚
靠近食物	+1	引导方向
远离食物	-1	防止绕路
原地不动	-0.1	鼓励行动
超时（防死循环）	-10	限制单局时长

5.4 DQN 网络结构

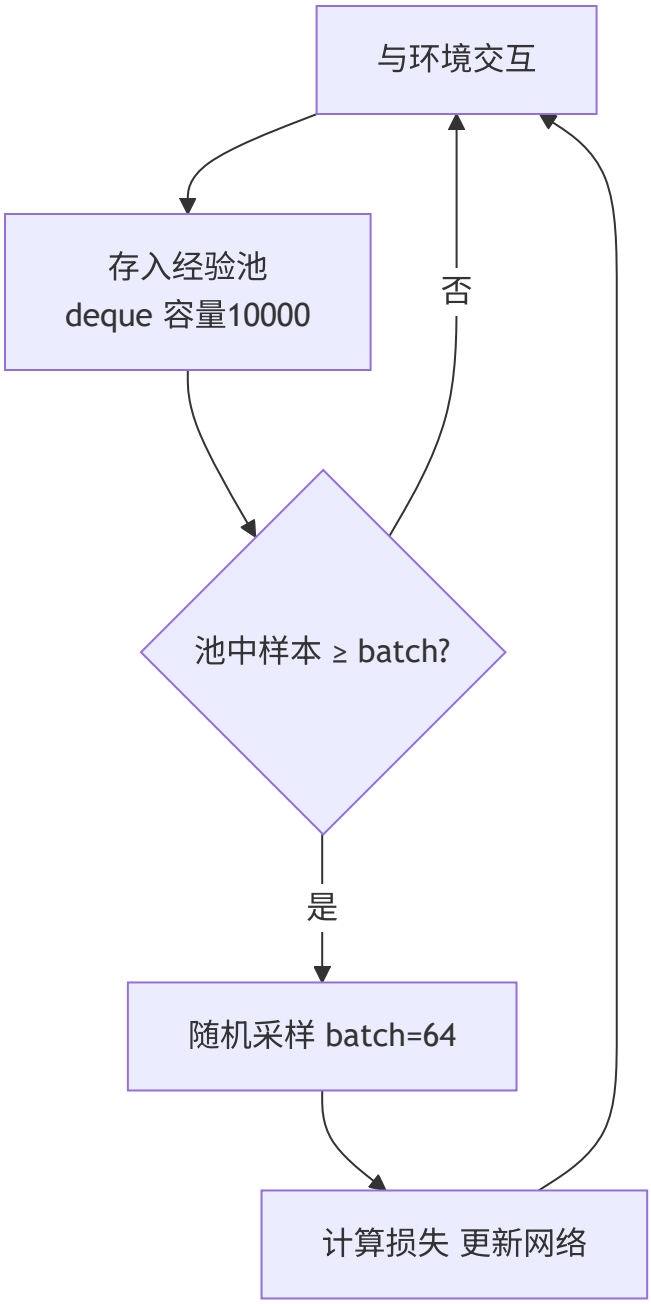
用三层全连接神经网络**拟合 Q 值函数**：输入状态（10 维），输出 4 个动作的 Q 值。




```
class DQN(nn.Module):
    def __init__(self, input_size, output_size):
        super().__init__()
        self.fc1 = nn.Linear(input_size, 64)
        self.fc2 = nn.Linear(64, 64)
        self.fc3 = nn.Linear(64, output_size)  # 输出 4 个动作 Q 值
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)
```

5.5 经验回放（ReplayMemory）

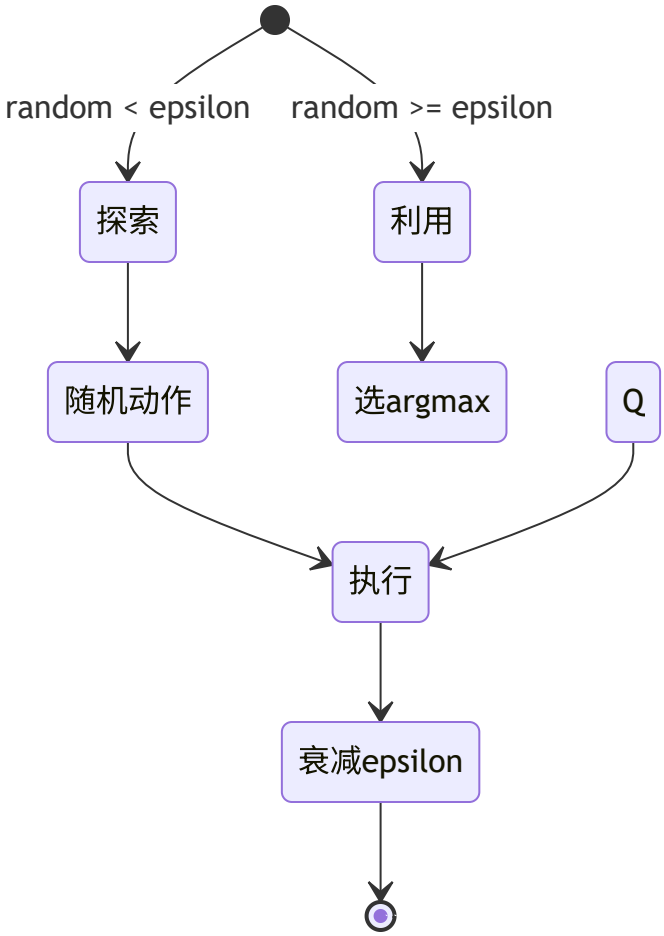
真实学习中，相邻样本高度相关，直接训练效果差。**经验回放**随机存储历史经验 `(s, a, r, s', done)`，训练时随机采样一批（batch），打破相关性、提升稳定性。



5.6 ε-贪婪策略（探索 vs 利用）

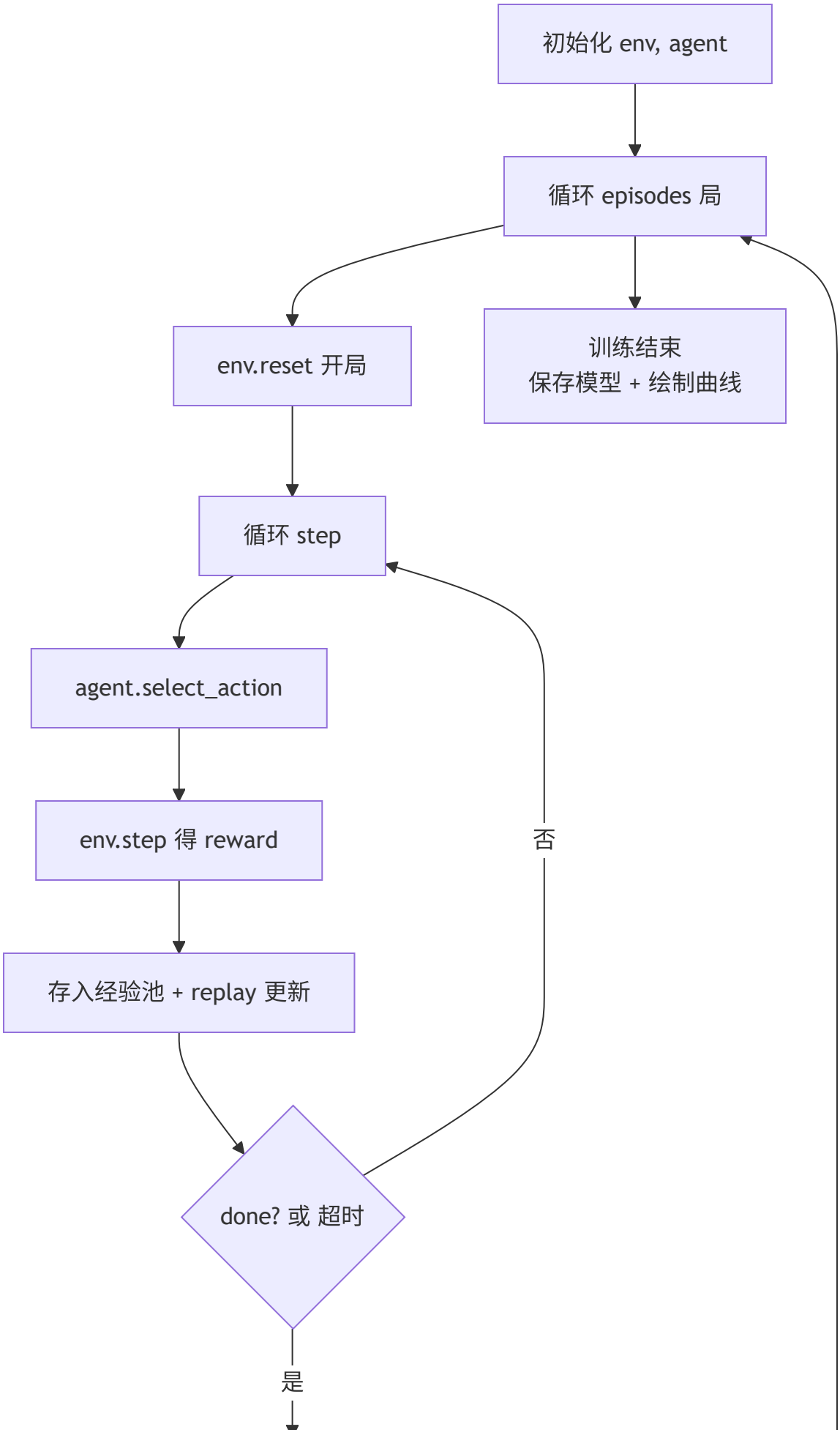
- **探索**：以概率 ε 随机选动作，发现新策略；
- **利用**：以概率 1-ε 选当前 Q 值最大的动作，用已有知识。

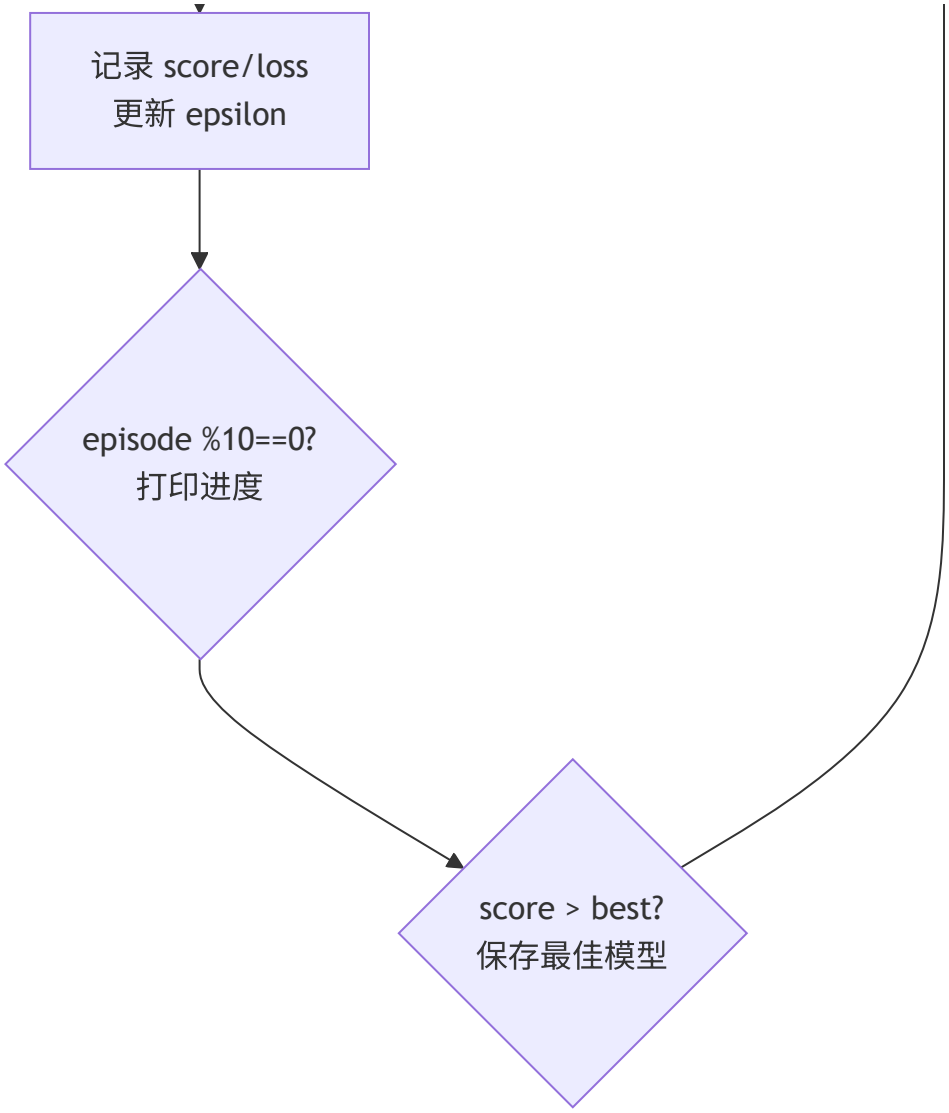
训练初期 ε 大（多探索），随训练衰减至 ε_end（多利用）： `epsilon = max(0.01, epsilon * 0.995)` 。



6. 训练流程设计

`train_agent()` 将环境、智能体、经验回放串联，完成完整训练闭环。


初始化 env, agent
循环 episodes 局
env.reset 开局
训练结束
保存模型 + 绘制曲线
循环 step
agent.select_action
env.step 得 reward
存入经验池 + replay 更新
done? 或 超时
否
是



关键机制：

- **双网络**： `policy_net` （训练）与 `target_net` （稳定目标）定期同步，防止 Q 值震荡；
- **目标 Q 值**： `target_q = reward + γ · max Q(s') · (1 - done)` ；
- **损失函数**： `MSE(current_q, target_q)` ，Adam 优化器更新。

> 📈 训练结束后自动绘制 **得分曲线** 与 **损失曲线** （ `training_progress.png` ），用于观察学习是否收敛。

7. 演示与运行

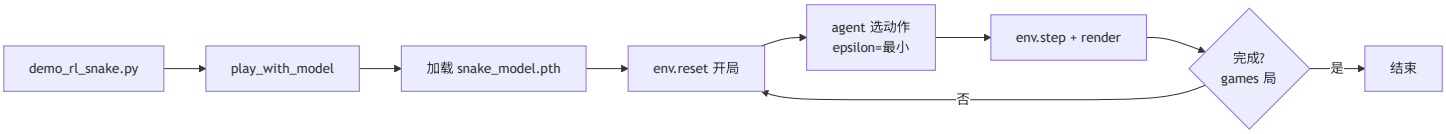
7.1 训练（完整流程）

`enhancedSnake.py` 的 `__main__` 中调用 `train_agent(episodes=2000)` ，依次完成训练并保存模型。

7.2 仅演示（快速查看 AI 效果）

`demo_rl_snake.py` 是**轻量入口**，加载已训练模型即可观看 AI 自动游戏：

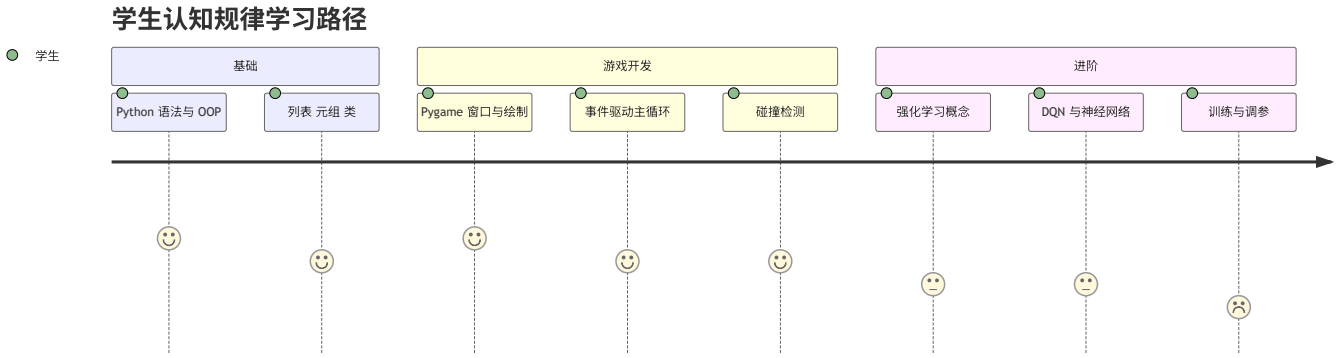
```
from enhancedSnake import play_with_model
if __name__ == "__main__":
    play_with_model(model_path="snake_model.pth", games=5, max_steps=500)
```



7.3 传统手动模式（扩展练习）

在 `step()` 中改用**键盘事件**决定 action，即可还原经典玩法——这正是"事件驱动"知识点的实践（参考 PDF 设计文档的键盘控制设计）。

8. 知识点总结与学习路径

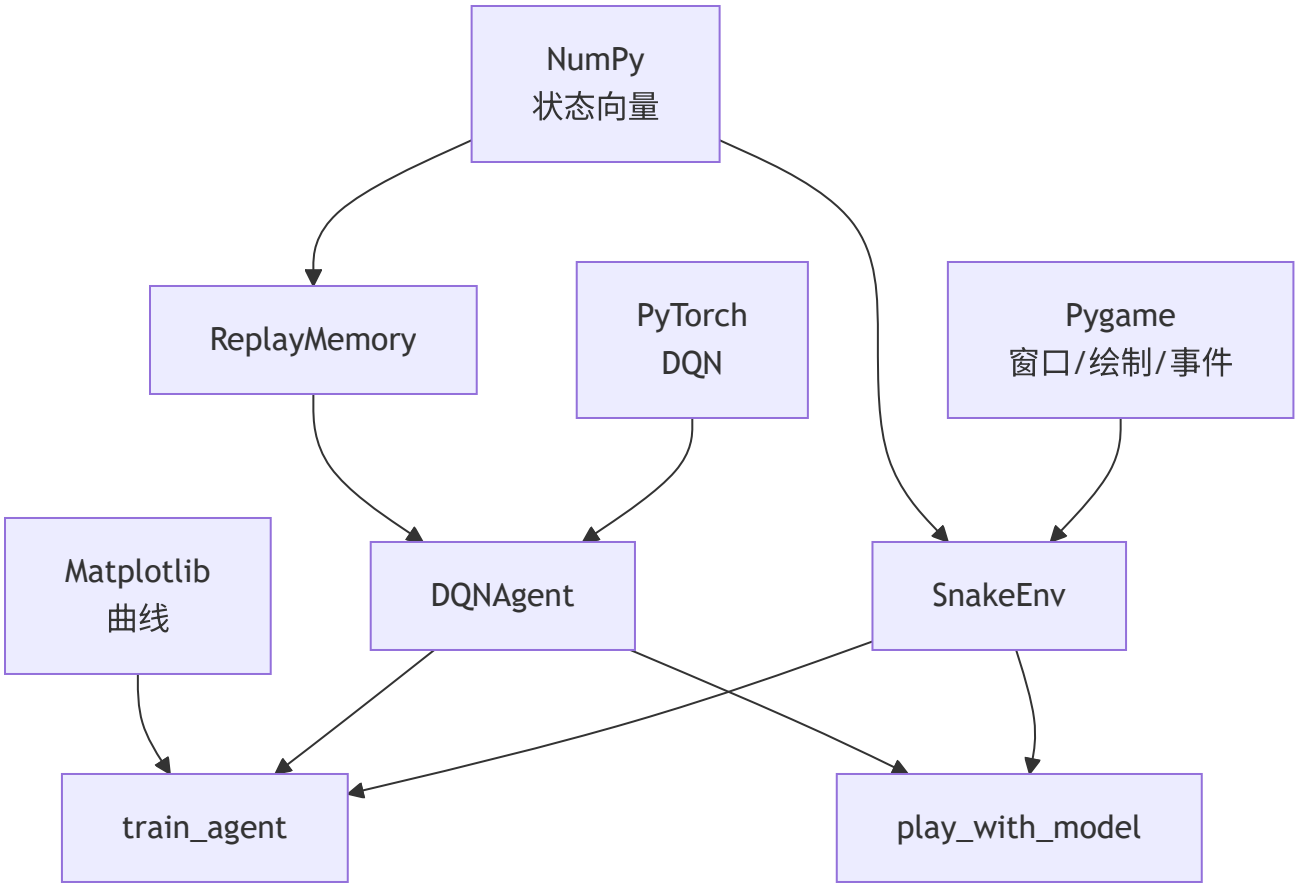


阶段	建议任务	对应章节
⭐ 入门	理解单文件结构、运行游戏、看懂主循环	1–4
⭐⭐ 进阶	拆分多文件、添加键盘控制、修改难度参数	2、4
⭐⭐⭐ 高阶	理解状态/奖励设计、DQN 原理、调参训练	5–7

扩展练习建议

1. **增加障碍物**：在状态中加入障碍物危险检测；
2. **改进状态**：加入"蛇身长度""路径规划"信息；
3. **改用 CNN**：用网格图像作为输入，替代手工特征；
4. **人机对战**：结合第 4 章键盘控制，实现"学生 vs AI"。

附录：模块依赖总览



> **结语**：本项目从经典贪吃蛇出发，逐步引入面向对象、事件驱动，最终进阶到深度强化学习，形成一条**由浅入深、学以致用**的完整学习链路。希望这份文档能成为你 Python 与 AI 学习路上的清晰路标 🚀。